

论文解构报告

Tally: Non-Intrusive Performance Isolation for Concurrent Deep Learning Workloads

Wei Zhao, Anand Jayarajan, Gennady Pekhimenko

ASPLOS '25 (30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems) · 2025

GPU sharing

performance isolation

deep learning systems

kernel scheduling

cloud infrastructure

源文件: <https://arxiv.org/pdf/2410.07381>

arXiv: 2410.07381

DOI: <https://doi.org/10.1145/3669940.3707282>

生成时间: 2026-07-29

目录

摘要翻译	3
1. 一句话总览	3
2. 阅读范围与证据状态	3
3. 根问题：论文究竟要解决什么	3
4. 旧方法为何不够	3
5. 核心洞见与假设	4
6. 方法：从洞见到可执行系统	5
7. 为什么它可能有效	8
8. 实验与指标：证据是否支撑主张	8
9. 图表解读与论证关系	10
10. 完整因果推理树	11
11. 结论、贡献与边界	11
12. 术语与第一性原理学习路径	11

摘要翻译

GPU 利用率不足是许多生产环境深度学习集群面临的重要问题，会导致作业排队时间延长和运营成本增加。GPU 共享是缓解该低效问题的一种有前景的方案，它允许多个工作负载在单块 GPU 上并发执行，从而提高资源利用率。然而，现有 GPU 共享机制存在集成成本高、性能隔离不足以及应用兼容性有限等缺陷，导致其在生产环境中的部署面临关键障碍。为解决这些问题，我们提出了 Tally，一种非侵入式的 GPU 共享机制，能够提供稳健的性能隔离和全面的工作负载兼容性。Tally 实现稳健性能隔离能力的关键在于其细粒度的线程块级 GPU 内核调度策略，该策略能有效缓解工作负载并发执行所带来的干扰。我们在多种工作负载上对 Tally 进行了评估，结果表明：当高优先级推理任务与尽力而为（best-effort）训练工作负载并发执行时，Tally 在推理任务第 99 百分位延迟上仅带来平均 7.2% 的开销，而当前最先进的 GPU 共享系统（如 TGS）的开销高达 188.9%；与此同时，Tally 的系统吞吐量达到了 TGS 的 80% 以上。

1. 一句话总览

论文元数据

字段	内容
标题	Tally: Non-Intrusive Performance Isolation for Concurrent Deep Learning Workloads
作者	Wei Zhao, Anand Jayarajan, Gennady Pekhimenko
发表场所 / 年份	ASPLOS '25 (第 30 届 ACM 国际编程语言与操作系统体系结构支持会议), 2025
研究领域	计算机系统 (GPU 资源管理 / 深度学习系统)
关键词	GPU sharing, performance isolation, deep learning systems, kernel scheduling, cloud infrastructure

资源链接

- 原始文件：本地上传文件（无外部链接）
- arXiv: <https://arxiv.org/pdf/2410.07381> (编号 2410.07381)
- DOI: <https://doi.org/10.1145/3669940.3707282>
- 代码 / 数据集：文中未说明

标签 (3-5 个)： GPU 共享 · 性能隔离 · 内核调度 · 深度学习系统 · 云基础设施

Tally 是部署在应用与 GPU 之间的非侵入式虚拟化层，通过对 GPU 内核做线程块（thread block，GPU 上可独立调度的最小执行单元，类比车间里彼此不依赖的独立工位）级别的"切片"与"抢占"代码变换，使高优先级推理任务与 best-effort 训练任务共享同一 GPU 时，前者的 P99 尾延迟平均仅增加 7.2%，远低于现有最优方案 TGS 的 188.9%，同时无需修改用户代码 [作者明确陈述][实验支持]。

2. 阅读范围与证据状态

- **OCR 覆盖范围：**文本证据覆盖论文第 1-6 节及部分附录内容，图表证据覆盖第 3、5、6、10、11、12 页共 3 批次 6 张图 (Figure 1、2、3、5、6、7)。
- **明显缺失或损坏：**P99 延迟、吞吐、系统吞吐等核心指标未给出可逐字定位的数学公式，仅有文字定义 [文中未给出公式编号]；论文与 MIG、REEF、AntMan、Orion 等多个基线缺乏统一定量对比实验，相关比较停留在定性讨论。
- **背景知识边界：**本报告中对 GPU 架构、线程块等概念的类比说明属于 [背景知识]，用于辅助理解，不代表论文原文表述。

3. 根问题：论文究竟要解决什么

生产级深度学习集群的 GPU 利用率长期偏低——阿里巴巴数据显示中位数仅 4.2% [作者明确陈述]。这一问题重要，是因为 GPU 采购成本极高 (Meta 两座数据中心各配数十万块 H100，估计成本达 5 亿美元) [作者明确陈述]，利用率不足直接转化为巨额浪费与作业排队延迟。

低利用率的难点来自两个相互独立的物理/资源约束：训练任务常受 CPU 端数据读取瓶颈限制、无法"喂饱"GPU；在线推理任务的 GPU 需求随实时请求率剧烈波动（峰值可达均值 50 倍），迫使运维方为保证服务质量而过度预留资源 [作者明确陈述]。GPU 共享（让多个任务并发使用同一 GPU）是候选解法，阿里巴巴仿真显示有效共享可平均减少 50%（高峰达 73%）所需 GPU [作者明确陈述]。

成功标准：一个可在生产环境落地的共享机制必须同时做到非侵入、强性能隔离、广泛工作负载兼容 [作者明确陈述]。

本文的 story：既有方案在"调度粒度"这一技术维度上失效——粒度太粗（迭代级、内核级）无法把高优先级任务的排队延迟压到毫秒级以下；本文的转折是发现"线程块级调度"可以在不依赖特定框架、不要求内核幂等的前提下非侵入地实现，从而同时满足三条标准。

4. 旧方法为何不够

论文在第 3 节归纳三类根本限制：集成成本高、缺乏隔离保证、依赖狭窄工作负载特征 **作者明确陈述**。以下按论文实际比较的路线逐项展开因果链：

Time-Slicing / MPS (NVIDIA 驱动层)

前提：原本解决"透明共享 GPU 资源、最大化利用率"的问题 **作者明确陈述**，依赖的前提是"尽量多派发内核"的调度策略。

本文指出的失效点

优先级无关设计导致高优先级任务排队, 实验中 P99 延迟开销分别为 252.3%/345.0%，MPS 尾延迟最高达独立运行的 20 倍 **实验支持**。



本文对应模块

Priority-aware 调度器 + 块级内核变换。

检验：Figure 5 端到端对比。

MPS-Priority

前提：在 MPS 上加入优先级控制 **作者明确陈述**。

本文指出的失效点

调度粒度仍为内核级（粗粒度），实验开销 195.5% **作者明确陈述**。



本文对应模块

块级调度（切片+抢占）。

检验：Figure 5、Table 1 粒度对比。

TGS

前提：专为 DL 工作负载设计，通过自适应速率控制在容器编排中实现隔离 **作者明确陈述**，前提是内核级调度足够应对干扰。

本文指出的失效点

对长内核工作负载（如 Whisper）退化严重，跨负载延迟退化波动 15.6%–751.7% **实验支持**。



本文对应模块

块级调度 + profile-guided 配置选择。

检验：Figure 5、Figure 6a/6b。

Salus（应用层）

前提：暴露训练迭代边界实现迭代级管理 **作者明确陈述**。

本文指出的失效点

调度粒度为迭代级（Table 1 中 turnaround ≈ 3 s），远高于亚毫秒级需求 [基于文中逻辑的推断，推理依据：Table 1 数值与原文"迭代级/内核级带来 2.5–760 $\times$ 开销"的整体陈述，未针对 Salus 单独重跑实验]。



本文对应模块

块级调度。

检验：Table 1 单一案例。

AntMan、EffiSha、Orion、REEF、GPES、Kernelet 等

前提：各自依赖训练模式可预测（AntMan）、需要源码访问（EffiSha、GPES）、需要离线 profiling（Orion）、需要内核幂等（REEF）等前提，均限制了通用性或非侵入性 **作者明确陈述**。

本文指出的失效点

限制了通用性或非侵入性 **作者明确陈述**。



本文对应模块

分别用任务无关设计、拦截设备代码而非源码、透明在线 profiler、不依赖幂等假设的统一同步+抢占变换回应。

检验：无法从当前 OCR 内容确认。

5. 核心洞见与假设

- **核心洞见（隔离效果由调度粒度决定）**：要达到毫秒级隔离，调度粒度必须细至块级或线程级，粗粒度调度无法满足 [作者明确陈述，Table 1 实验支持]。
- **核心洞见（块级调度可非侵入且通用地实现）**：GPU 编程模型保证同一内核内的线程块彼此独立、调度顺序不确定，这一硬件特性使块级调度不像线程级调度那样依赖"幂等"假设，因而可对所有内核通用适用 [作者明确陈述]。
- **经验判断（块级调度"足够好"）**：线程级调度理论隔离更强，但作者认为块级调度对多数实际场景已足够——这是一个未经严格定量证明的经验判断，实验部分以整体结果间接支持 [作者明确陈述，间接实验支持]。

- **实现落地思路**: 切片与抢占可通过对内核设备代码 (PTX) 的变换实现, 设备代码经拦截设备 API 调用获得, 无需用户配合 [作者明确陈述]。
- **隐含假设**: 拦截到的内核代码可被安全反汇编、修改并重新编译执行; 论文未讨论该假设何时可能失效 (如加密/混淆内核) [基于文中逻辑的推断, 推理依据: 4.3 节描述的实现路径]。

6. 方法: 从洞见到可执行系统

Tally 采用客户端-服务端架构: 客户端库通过 LD_PRELOAD (Linux 提供的函数调用拦截机制, 类似给程序"加装外置行车记录仪"而不拆改发动机) 拦截应用的 GPU API 调用, 转发给独立 server 进程; server 对内核做变换、profiling 并统一调度后交给 GPU 执行 [作者明确陈述]。

1. **切片变换**: 动机是需要一种轻量方式把大内核拆成可控小单元, 便于调度器细粒度插入高优先级任务 [作者明确陈述]。做法是把原内核 (如 16 个线程块) 拆成多个子内核 (如 4 个各 4 块), 新增 offset 参数把 blockIdx 替换为 offset+原blockIdx 以恢复真实块索引 [作者明确陈述]。优势是实现简单、语义天然不变; 代价是多次内核启动带来额外开销 (并入 5.7 节"内核变换平均 25% 开销", 未拆分切片专属占比) [文中未说明该拆分]。
2. **抢占变换**: 动机是切片仍可能有排队延迟, 需要能中断正在执行的内核 [作者明确陈述]。借鉴 Persistent Thread Block 范式, 发射固定数量 worker 线程块, 通过全局计数器迭代领取原始 block 的工作, 并检查"抢占标志"以便随时中断、之后恢复 [作者明确陈述]。优势是只需一次启动; 代价是需额外控制流与同步开销 [作者明确陈述]。
3. **统一同步变换**: 动机是抢占把 return 改为跳转指令, 可能导致同一 block 内线程在不同同步点调用同步指令 (同步分歧, GPU 编程模型下的未定义行为, 会导致内核无限停滞) [作者明确陈述]。做法是将所有同步点与 return 重定向到一个统一同步点, 检测是否全部线程到达返回状态, 未到达则循环回退, 直至全部对齐 [作者明确陈述]。这是抢占变换能自动安全应用于所有内核的前置保障 [作者明确陈述]。
4. **透明分析器 (Profiler)**: 动机是切片/抢占的性能权衡因内核而异, 参数需按内核调优, 人工静态设定不可行 [作者明确陈述]。对候选配置实测或估算 turnaround latency (切换延迟, 指 best-effort 内核释放资源所需的估计时间, 类比"占用会议室的人被通知后需要多久才能腾出房间"):

$$turnaround_latency = \frac{kernel_latency \cdot worker_blocks}{total_blocks}$$

用于抢占配置的估算 (kernel_latency: 内核整体执行时间; worker_blocks: 分配 worker 数; total_blocks: 总线程块数), 选出满足阈值 (默认 0.0316ms) 且性能最优的配置, 按内核复用不重复 profiling [作者明确陈述, 原文公式]。

5. **Priority-aware 调度器**: 动机是需要总控算法整合前述能力, 兼顾高优先级即时响应与 best-effort 吞吐 [作者明确陈述]。策略是优先立即派发高优先级内核 (若有 best-effort 内核在执行则抢占之), 仅在高优先级空闲时以可分片/可抢占形式调度 best-effort 内核 [作者明确陈述]。反应式设计的代价是若高优先级请求恰好在 best-effort 内核刚启动时到达, 仍需等待完成或抢占 [作者明确陈述]。

6. **非侵入虚拟化层**: 整合以上模块的运行载体, 用共享内存通信与本地状态缓存降低通信开销 (5.7 节报告虚拟化层平均开销仅 1%) [作者明确陈述][实验支持]。

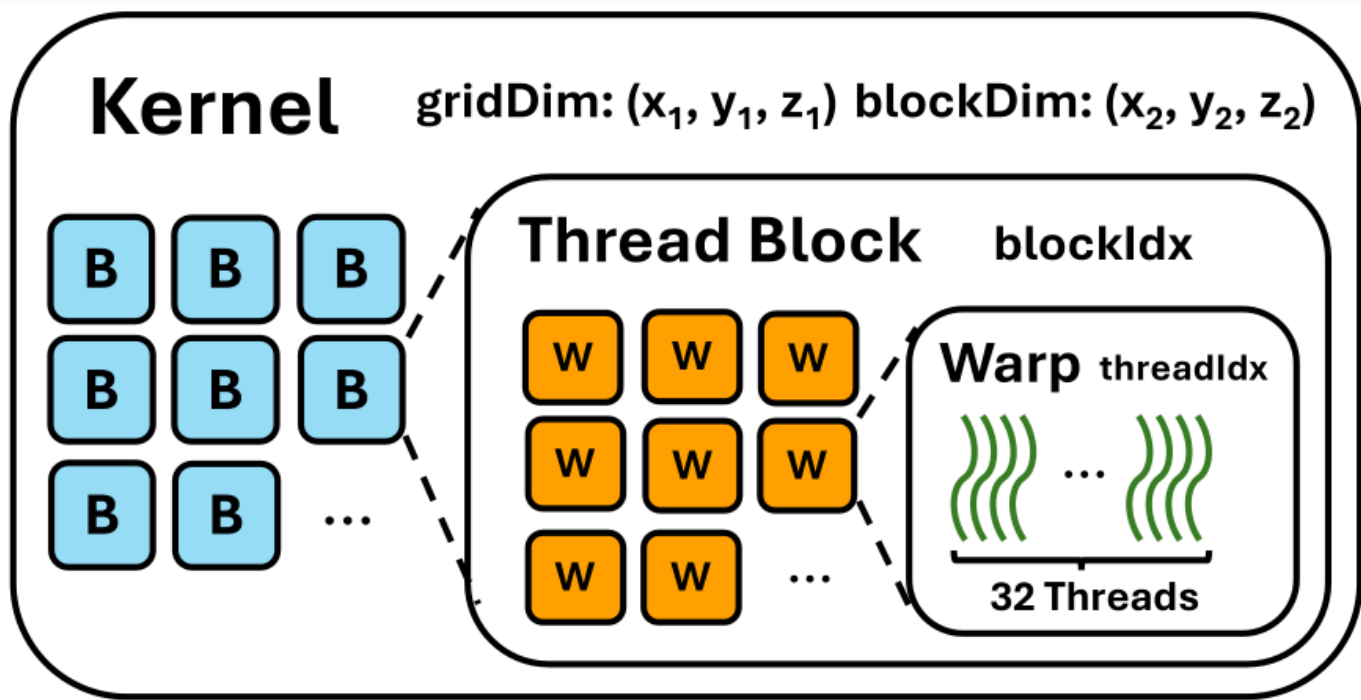


Figure 1. Overview of the GPU programming model.

图组解读

- **图组结论**: Kernel 由 Block 网格构成, Block 含 Warp, Warp 固定 32 Thread。
- **论证关系**: 支撑线程块独立特性, 为切片变换重构索引提供结构前提。
- **适用范围**: 通用 GPU 编程模型层级示意, 未可视化调度顺序不确定性。

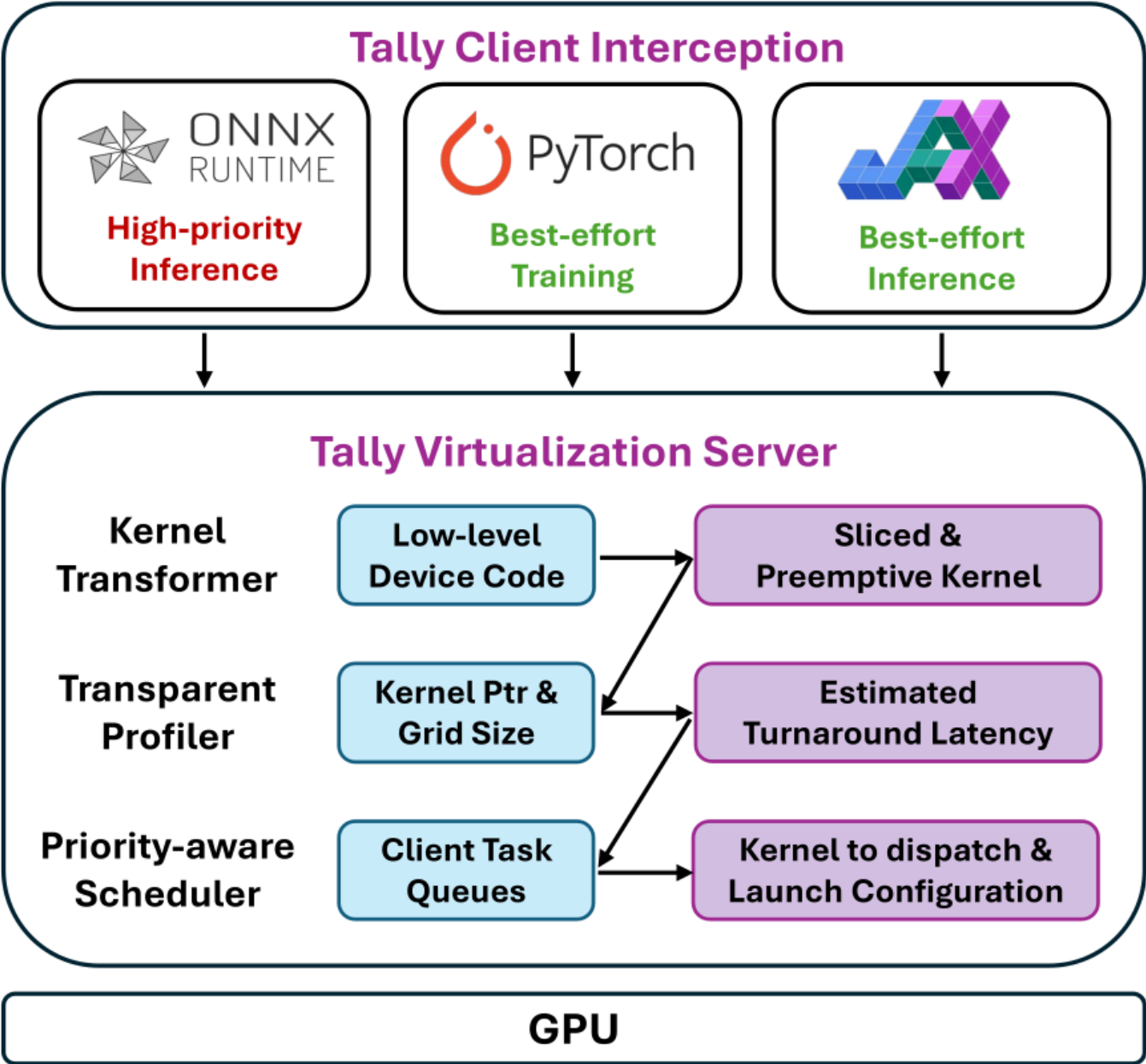


Figure 2. Tally architecture.

图组解读

- **图组结论**：多类 Client 经拦截层进 Server，Server 内变换器、分析器、调度器依次处理并连 GPU。
- **论证关系**：呈现非侵入虚拟化层机制，印证 Client 拦截与 Server 统一调度的流水线。
- **适用范围**：通用架构示意，图中 JAX 图标与实际评测框架不完全一致。

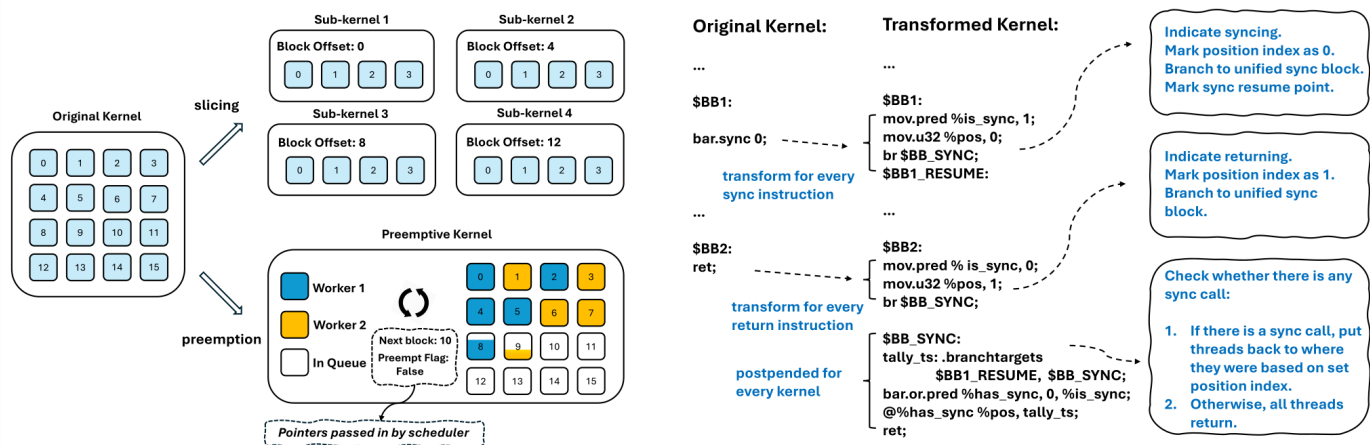


Figure 3. Kernel transformations: (a) slicing and preemption transformations (b) unified synchronization transformation.

图组解读

- 图组结论：**切片按 Offset 拆分子内核；抢占由 worker 领任务；同步重定向至统一检查点。
- 论证关系：**展示 PTX 代码变换机制，证明切片与抢占可非侵入实现且统一同步可消除分歧。
- 适用范围：**示意性示例，未证明在 Cooperative Groups 等内核上的普适性。

7. 为什么它可能有效

- 块级调度的粒度足够细（微秒级 turnaround），预期能把高优先级任务的等待时间压到接近其内核本身执行时间的量级，前提是 profiler 给出的配置选择准确 [基于文中逻辑的推断]。
- 统一同步变换消除同步分歧风险，使抢占可以自动应用于任意内核而非仅限特定代码模式，预期收益是通用性提升，条件是内核不使用跨块协作原语（如 Cooperative Groups）[基于文中逻辑的推断]。
- Priority-aware 调度器的“严格让路”策略预期在低负载下几乎不牺牲吞吐，但在高负载（低 idle）区间会以吞吐换隔离，这一权衡的存在条件是 best-effort 任务数量足够多导致资源竞争 [基于文中逻辑的推断，实验部分间接支持]。

8. 实验与指标：证据是否支撑主张

8.1 实验问题与判定标准

- 效果：**Tally 相对基线能否显著降低高优先级任务尾延迟开销同时保持有竞争力吞吐？对应 Figure 5 端到端对比。
- 必要性：**块级内核变换相对于“仅调度算法”是否是隔离效果的必要条件？对应 5.5 节性能分解消融。
- 鲁棒性：**隔离能力是否对流量负载、训练任务配对具有稳定性？对应 5.3 节负载敏感性。
- 适用范围：**能否扩展到多个 best-effort 任务？对应 5.4 节可扩展性。
- 参数敏感性：**turnaround latency 阈值如何影响延迟-吞吐权衡？对应 5.6 节。

8.2 数据、任务、对比与运行设置

- 硬件/环境：**AWS p4d.24xlarge 实例（单张 A100 GPU）[作者明确陈述]。
- 工作负载：**6 个训练 + 6 个推理基准，覆盖 BERT、Llama-2-7B、ResNet50、YOLOv6M、GPT-Neo-2.7B、Stable Diffusion（推理）及 Whisper、ResNet50、PointNet、Pegasus、GPT2、BERT（训练）[作者明确陈述]。
- 流量模拟：**MAF2（微软生产流量轨迹）驱动的推理请求模拟 [作者明确陈述]。
- 基线：**Time-Slicing、MPS、MPS-Priority、TGS 四个系统 [作者明确陈述]。
- 推理引擎/训练框架：**ONNX Runtime/TorchInductor/Hidet（推理）、PyTorch（训练）；架构图（Figure 2）中出现的 JAX 未见于实际评测框架列表 [基于文中逻辑的推断]。
- 重复次数/统计方式：**文中未说明。
- 超参数：**turnaround latency 阈值默认 0.0316ms，经 5.6 节扫描确定 [作者明确陈述]。

8.3 指标字典与对应公式

- 指标与方向：**P99 延迟（99th percentile latency），测量推理任务响应时间，单位 ms，越低越好。
 - 定义与公式：**来源层级为无可核验公式——论文仅说明其为“评估推理任务性能的主要指标”，未给出统计计算式（如采样窗口、聚合方式）[文中未给出公式编号]。
 - 实验用途：**端到端结果（5.2）、负载敏感性（5.3）、性能分解（5.5）。
 - 读数与边界：**Tally 平均开销 7.2% vs. Time-Slicing/MPS/MPS-Priority/TGS 的 252.3%/345.0%/195.5%/188.9% [实验支持]；不能证明该结论在非 A100 硬件上的成立性。
- 指标与方向：**吞吐（Throughput），单位时间处理样本数，越高越好。

- **定义与公式**: 来源层级无可核验公式——作者文字定义"the number of samples processed per unit of time", 无数学式 [作者明确陈述]。
- **实验用途**: 各实验的吞吐对比。
- **读数与边界**: 无独立数值, 通常与系统吞吐一并报告。
- **指标与方向**: 系统吞吐 (System Throughput), 多并发工作负载整体吞吐, 越高越好。
 - **定义与公式**: 来源层级无可核验公式——作者文字定义为"各任务归一化吞吐求和", 未说明归一化基准 [作者明确陈述]。
 - **实验用途**: 5.2/5.3/5.4 系统吞吐对比。
 - **读数与边界**: Tally 达四基线的 105.2%/83.6%/80.6%/80.3% [实验支持]; 10% 空闲时 Tally=1.19 vs. TGS=1.65 (严格优先级策略牺牲部分吞吐) [实验支持]。
- **指标与方向**: Turnaround latency (切换延迟), best-effort 内核释放资源所需估计时间, 单位 ms/ μ s, 越低越好 (隔离更强)。
 - **定义与公式**: 通用定义来源层级无可核验公式 (文字定义, 无通用数学式); 抢占内核专用估算式来源层级为原文公式 (4.2 节正文给出, 未标注 Eq. 编号): $turnaround_latency = \frac{kernel_latency_worker_blocks}{total_blocks}$ [作者明确陈述]。
 - **实验用途**: 说明调度粒度对隔离的影响 (Table 1); profiler 选优的核心依据。
 - **读数与边界**: Table 1 实测 (BERT vs. Whisper, A100): 迭代级 $\approx$ 3s、内核级 $\approx$ 10ms、线程块级 $\approx$ 304 μ s、线程级 $\approx$ 38 μ s, BERT 单次推理仅需 3.93ms [实验支持]; 仅一组代表性案例, 非系统性扫描。
- **指标与方向**: Turnaround latency 阈值, 调度器接受的上限, 单位 ms, 需权衡 (越低隔离越强、吞吐可能下降)。
 - **定义与公式**: 来源层级无可核验公式——默认值 0.0316ms 为 5.6 节敏感性扫描 (0.01ms–10ms 六档) 的经验调参结果, 非公式推导 [作者明确陈述]。
 - **实验用途**: 4.2 节调度算法参数、5.6 节敏感性分析。
 - **读数与边界**: 阈值升高, 延迟增加 (尤其 Whisper 等长内核)、吞吐仅小幅提升 [实验支持]; 未说明该值随 GPU 型号的迁移性 [文中未说明]。
- **指标与方向**: 空闲时间/负载 (idle time / load), GPU 非服务时间占比, 0–100%。
 - **定义与公式**: 来源层级无可核验公式——文字定义 load 为"推理服务实际处理请求所占时间百分比", idle=100%–load。
 - **实验用途**: 5.3 节负载敏感性分析的自变量。
 - **读数与边界**: 覆盖 10%–90% 区间 [实验支持]。

8.4 主结果、消融与证据边界 比较工作与被批判限制的呼应

比较工作/基线	核心限制或失效条件	本文对应模块	直接实验与仍未验证的边界
Time-Slicing / MPS	优先级无关, "尽量多派发"导致排队, 开销 252.3%/345.0% [实验支持]	Priority-aware 调度器	Figure 5 已验证; 未验证非 A100 硬件
MPS-Priority / TGS	内核级粒度不足, 开销 195.5%/188.9%, TGS 延迟退化跨组合波动 15.6%–751.7% [实验支持]	块级调度 (切片+抢占) +profile-guided 配置	Figure 5/6 已验证; 未验证异构大规模场景
REEF / AntMan / Orion / MIG	分别要求幂等、局限训练场景、需离线 profiling、静态粗粒度分区 [作者明确陈述]	统一同步+抢占变换 / 任务无关设计 / 透明在线 profiler / 动态资源分配	无直接定量对比实验, 仅定性讨论 [无法从当前 OCR 内容确认]

- 比较结果仅说明本文实验设置内的相对表现, 不能证明基线方法在其他场景下必然失效。

主张与证据表

主张	直接证据 (实验/表图)	证据强度与边界
Tally P99 延迟开销均值 7.2%, 远低于 TGS 等基线	摘要、5.2 节、Figure 5	[实验支持], 覆盖 36 种推理 $\times$ 训练组合
块级/线程级调度是亚毫秒隔离的必要条件	Table 1	[实验支持], 仅 BERT+Whisper 单一案例, 非系统扫描
块级调度对隔离是必要的 (相较仅调度算法)	5.5 节, Figure 7b: No-scheduling 最差 30 $\times$ 延迟放大, 仅调度算法在 Whisper/BERT 仍达 10 $\times$, 完整 Tally 平均 4.0% (最差 6.2%) [实验支持]	消融实验直接支撑必要性主张
Tally 隔离能力具有鲁棒性	5.3 节, Figure 6a/6b	仅覆盖 BERT、Llama-2-7B 两个推理代表
可扩展到多个 best-effort 任务	5.4 节, Figure 7a	仅用同构 ResNet50 模拟, 未验证异构场景 [基于文中逻辑的推断]
Tally 优于 MIG	第 6 节讨论	[作者明确陈述], 无实验数值支持, 定性论证
虚拟化层开销约 1%	5.7 节	[实验支持], 未说明是否分推理/训练拆分 [文中未说明]

- 关键设置提示: 所有主结果均基于单张 A100、固定 50% 负载或指定 idle time 区间, 跨硬件/跨负载泛化性未被验证。

· Cooperative Groups 类内核未被纳入评测基准，相关适用范围主张缺乏实证 [作者明确陈述]。

9. 图表解读与论证关系

表格证据: Table 1. Comparison of BERT inference latency against turn-around latency of different scheduling granularity for Whi... 对比基线 呼应：第 4 节：调度粒度对切换延迟的影响

- **图组结论**：BERT 推理配 Whisper 训练时，迭代级切换延迟约 3s，内核级约 10ms，块级降至 304μs。
- **论证关系**：对比不同调度粒度切换延迟，论证块级调度实现亚毫秒隔离的必要性。
- **适用范围**：仅为 BERT+Whisper 在 A100 上的单一代表性案例。

Inference time (BERT)	Turnaround latency (Whisper)			
	Iteration	Kernel	Block	Thread
3.93ms	~ 3s	~ 10ms	~ 304μs	~ 38μs

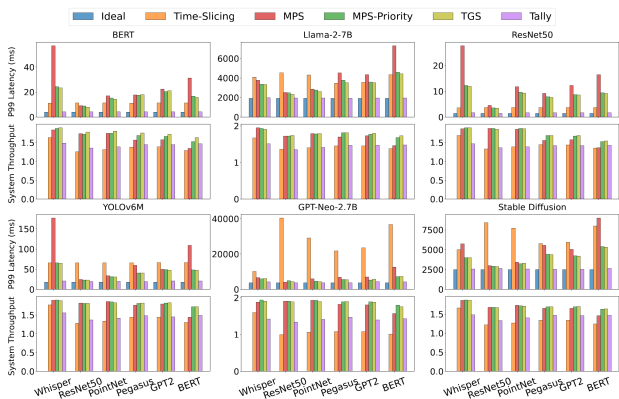


Figure 5. Analysis of 99th-percentile latency and system throughput across different high-priority inference and best-effort training workload combinations under various GPU sharing mechanisms.

图组解读 对比基线 呼应：第 8 节：端到端延迟与系统吞吐结果

- **图组结论**：36 种推理与训练配对下，Tally 的 P99 延迟最接近 Ideal，系统吞吐与基线相当。
- **论证关系**：验证 Tally 隔离效果显著优于 Time-Slicing、MPS 和 TGS 等基线。
- **适用范围**：覆盖单张 A100 与 50% 负载，未证明消融占比。

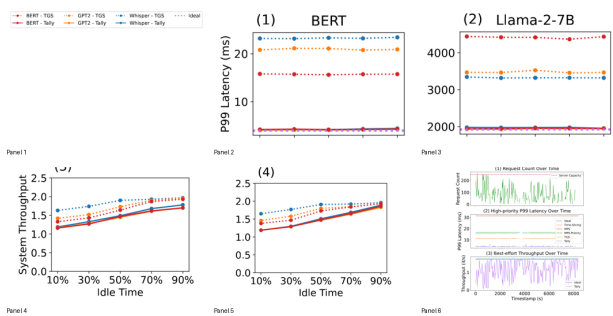


Figure 6. (a) Latency and throughput for high-priority BERT and Llama2-7B inference co-located with BERT, GPT-2, and Whisper training under Tally and TGS across different traffic loads. (b) Time-series visualization of user traffic, tail latency, and throughput over time for BERT inference co-located with BERT training across different GPU sharing systems.

图组解读 实验结论 呼应：第 8 节：流量敏感性与鲁棒性

- **图组结论**：Tally 延迟在 10%–90% 空闲区间贴近 Ideal，低空闲时牺牲吞吐换隔离。
- **论证关系**：证明 Tally 隔离对流量与训练配对具鲁棒性，印证 Priority 策略。
- **适用范围**：仅覆盖 BERT 和 Llama-2-7B 推理代表，部分曲线辨读受限。

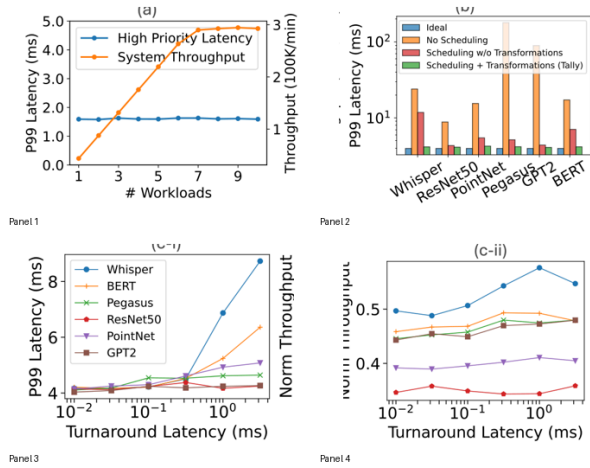


Figure 7. (a) Scalability performance of Tally with respect to number of concurrent workloads. (b) Performance decomposition of Tally for BERT inference. (c) Latency and throughput under different turnaround latency threshold settings.

10. 完整因果推理树

- 根问题：生产级 DL 集群 GPU 利用率低（中位数 4.2%），如何在共享 GPU 时同时保证利用率与高优先级任务性能？
 - 旧方法限制：集成成本高、粒度粗导致隔离不足（迭代级/内核级 turnaround 达 3s/10ms）、依赖狭窄工作负载特征（幂等/可预测模式）
 - 核心洞见/假设：块级调度可在不依赖幂等假设的前提下非侵入通用实现，且隔离效果足够
 - 方法模块：切片变换、抢占变换、统一同步变换、透明 Profiler、Priority-aware 调度器、非侵入虚拟化层
 - 可验证预测：块级变换可把 turnaround latency 压至微秒级，从而使高优先级延迟接近独立运行水平
 - 指标与实验：P99 延迟、系统吞吐、性能分解消融（5.2/5.5 节）
 - 图表证据：Figure 5（端到端）、Figure 7b（消融）
 - 结论与适用边界：36 种组合下延迟开销均值 7.2%、必要性获消融支持；但未验证异构大规模、非 A100 硬件及 Cooperative Groups 内核场景 [实验支持]

11. 结论、贡献与边界

- 贡献总结**：提出非侵入式 GPU 共享系统 Tally；提出任务无关的块级内核变换（切片+抢占+统一同步）与 priority-aware 调度器；在多样化工作负载上验证其吞吐可与最优基线持平同时保证强隔离 [作者明确陈述]。
- 证据充分的结论**：Tally 在本文测试的 36 种推理-训练组合下显著优于四个非侵入基线的尾延迟开销（7.2% vs. 188.9% 起），且块级变换对隔离是消融验证过的必要因素 [实验支持]。
- 尚属推断的意义**：块级调度“足够好”（相对线程级调度）为经验判断，未有严格定量证明 [基于文中逻辑的推断]。
- 适用条件**：面向“一个高优先级任务 + 多个 best-effort 任务”场景，多高优先级任务并发竞争的情形未讨论 [作者明确陈述]。
- 局限与下一步验证**：与 MIG/REEF/AntMan/Orion 缺乏统一定量对比；核心指标缺精确公式；Cooperative Groups 内核未实测；异构大规模 best-effort 扩展性未验证；阈值 0.0316ms 跨硬件迁移性未讨论。

审稿式风险检查：

- 贡献新意：块级调度思路借鉴 EffiSha/GPES 等已有工作，本文创新点在于非侵入实现（拦截设备代码而非源码）及统一同步变换，新意主要体现在工程实现层面而非全新调度原语 [基于文中逻辑的推断]。
- 写作/可复现性：代码/数据集链接文中未说明，复现依赖细节（如统计口径、重复次数）未披露 [文中未说明]。
- 效果大小：主结果建立在单一硬件（A100）与固定基准套件上，效果量级是否随硬件/规模变化，当前证据未显示该风险已被评估。
- 实验覆盖：与多个关键基线（MIG、REEF、AntMan、Orion）缺乏定量对比，是本文证据链中最明显的覆盖缺口。
- 方法设计：隐含假设“内核代码可安全拦截并重编译”未讨论失效边界（如加密/混淆内核），当前证据未显示该假设已被压力测试 [基于文中逻辑的推断]。

12. 术语与第一性原理学习路径

12.1 核心术语

- 术语**：线程块（Thread Block）——GPU 内核中可独立调度、彼此不依赖的最小执行单元集合。

- **第一性原理角色**: 受 GPU 编程模型约束——同一内核内的线程块执行顺序不确定、相互独立, 这是块级调度能非侵入通用实现的物理前提。
- **本文位置**: 切片变换通过 `offset+blockIdx` 操作线程块索引实现拆分。
- **术语**: Turnaround latency (切换延迟) ——best-effort 内核释放已占用 GPU 资源所需的估计时间。
 - **第一性原理角色**: 受调度粒度约束——粒度越粗, turnaround 越长, 直接决定隔离上限。
 - **本文位置**: Table 1 用于论证粒度选择; Profiler 用其估算式选择调度配置。
- **术语**: 切片 (Slicing) ——把一个大内核拆分为多个可独立启动的小子内核。
 - **第一性原理角色**: 通过增加内核启动次数换取更细的抢占插入点, 受"内核启动开销"这一资源约束。
 - **本文位置**: 模块一, 4.1 节。
- **术语**: 抢占 (Preemption) ——允许正在执行的内核被中断并在之后恢复。
 - **第一性原理角色**: 通过引入循环控制流和状态标志, 把"任务能否被打断"从内核启动边界下沉到迭代边界, 受同步安全约束 (需配合统一同步变换)。
 - **本文位置**: 模块二, 4.1 节。
- **术语**: 统一同步变换 (Unified Synchronization Transformation) ——将内核中所有同步点和返回点重定向到单一检查点的代码变换。
 - **第一性原理角色**: 解决"同步分歧"这一 GPU 编程模型下的未定义行为约束, 是抢占变换安全生效的前提条件。
 - **本文位置**: 模块三, 4.1 节。
- **术语**: Profile-guided 调度配置——通过运行时实测/估算为每个内核选择最优调度原语与参数。
 - **第一性原理角色**: 在"隔离强度"与"best-effort 吞吐"两个约束目标之间做权衡搜索。
 - **本文位置**: 模块四, 4.2 节。
- **术语**: 非侵入 (Non-intrusive) ——不需修改用户代码/框架即可生效的系统设计属性。
 - **第一性原理角色**: 约束来自生产集群"对运行中工作负载语义无感知"的现实条件, 决定了系统只能在设备代码/API 层面介入。
 - **本文位置**: 整体设计目标之一, 贯穿第 4 节。
- **术语**: P99 延迟——第 99 百分位响应时间, 衡量最坏情况用户体验。
 - **第一性原理角色**: 连接"任务调度延迟"与"服务质量 (SLA) 达标"两个基本量。
 - **本文位置**: 核心评估指标, 贯穿第 5 节。

12.2 学习路径

1. GPU 的 SM/线程/warp/线程块/网格层级结构——理解"哪一级别可被独立调度"是理解块级调度可行性的基础。
2. GPU 共享与性能隔离的基本目标——理解"为什么要在同一 GPU 上跑多个任务却互不拖累", 类比多辆车共用一条车道但要保证急救车优先。
3. 调度粒度与切换延迟的关系——理解"粒度越粗、切换等待越久"这一因果链, 是 Table 1 论证的基础。
4. GPU 内核编译中间表示 (PTX) 与设备代码拦截机制——理解 Tally 如何在不接触源码的情况下修改内核行为。
5. 优先级调度与 reactive 策略——理解"高优先级优先、必要时抢占"的调度决策逻辑, 是理解 Priority-aware 调度器权衡的前提。